

measure the accurate running time by doing so.

I am providing a Merge sort algorithm here, so that you can refer to and trace the difference in both codes, and see how many recursions and function calls are involved:

```
/* Function to merge the two halves arr[l..m] and
arr[m+1..r] of array arr[] */
void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    /* create temp arrays */
    int L[n1], R[n2];

    /* Copy data to temp arrays L[] and R[] */
    for(i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for(j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
    /* Merge the temp arrays back into arr[l..r]*/
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    /* Copy the remaining elements of L[], if there are any
*/
    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }
    /* Copy the remaining elements of R[], if there are any
*/
    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

```
k++;
}
}
/* l is for left index and r is right index of the sub-array
of arr to be sorted */
void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l+(r-l)/2;
        //Same as (l+r)/2, but avoids overflow for large l and h
        mergeSort(arr, l, m);
        mergeSort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}
```

Finally, as every algorithm has some pros and cons according to the input provided, some of the advantages and disadvantages of this new sorting algorithm are listed here.

Advantages

- Time efficient about $O(n)$ in even the worst case, and has sorting flexibility
- Elegant and lucid code that is easy to understand
- Requires only two scans to get the result, and it is free from recursion and extra overhead
- Is an advantage in many problems that use sorting as pre-processing to arrive at the result

Disadvantages

- It requires extra space as per input provided
- Wastes much of the space if the input is short and has a large number at any point, e.g., {5, 1, 11, 395, 159, 64, 9,}, though situations like this rarely occur

Applications of this sorting algorithm

There are several applications and uses that require sorting as pre-processing before doing real computation to get the result, which takes $O(n \log n)$ as the total running time. This particular problem can be decreased to a time of $O(n)$ with the help of this new sorting algorithm. Some examples can be:

- Finding the majority element
- Finding the triplet in an array
- Finding two elements whose sum is closest to zero. 

By: Santosh Kumar

The author is an alumnus of NIT-Jamshedpur with a computer science and engineering background. He loves playing around with algorithm design and programming. He currently works for Cisco Systems as a software engineer, and can be contacted at santoshnitsr2010@gmail.com